DevOps Command Cheat Sheet

Commands only - with placeholders and examples. Added Lab 3.3.11, Lab 6.2.7, and Lab 6.3.6 commands.

Placeholder rule: Anything inside < > is the part that may change, like <branch-name>, <container-name>, <port>, <username>, <gateway-ip>, and <repository-url>.

Linux / Folder Commands

COMMAND	EXAMPLE
`pwd` show current folder	`pwd`
`ls` list files and folders	`ls`
`cd <folder-path>` move into another folder	`cd ~/sample-app`
`mkdir <folder-name>` create one folder	`mkdir tempdir`
`mkdir -p <folder>/<subfolder>` create parent + subfolders if missing	`mkdir -p ~/sample-app/templates ~/sample-app/static`
`cp <file> <destination>` copy a file	`cp sample_app.py tempdir/.`
`cp -r <folder>/* <destination>` copy a folder contents	`cp -r templates/* tempdir/templates/.`
`rm <file-name>` remove a file	`rm oldfile.txt`
`rm -rf <folder-name>` force remove a folder and everything inside	`rm -rf tempdir`
`cat <file-path>` show file contents in terminal	`cat /var/jenkins_home/secrets/initialAdminPassword`

COMMAND	EXAMPLE
`grep "<text>"` find matching text	`grep "You are calling me from"`
COMMAND	**EXAMPLE**
`which <command>` show where a command is installed	`which docker`

Git / GitHub Commands

COMMAND	EXAMPLE
`git config --global user.name "<your-name>"` set Git username	`git config --global user.name "Iszy"`
`git config --global user.email "<your-email>"` set Git email	`git config --global user.email "iszy@example.com"`
`git init` turn current folder into a Git repository	`git init`
`git clone <repository-url>` download an existing repository	`git clone https://github.com/<your-username>/sample-app.git`
`git status` check changed / staged / untracked files	`git status`
`git diff` compare file changes	`git diff`
`git add <file-path>` stage one file	`git add sample_app.py`
`git add .` stage all changed files	`git add .`
`git commit -m "<message>"` save staged changes	`git commit -m "Initial sample app on port 5050"`
`git remote add origin <repository-url>` connect local repo to GitHub repo	`git remote add origin https://github.com/<your-username>/sample-app.git`
`git push -u origin <branch-name>` first push and set upstream branch	`git push -u origin main`
`git push origin <branch-name>` upload commits to GitHub	`git push origin main`

COMMAND	EXAMPLE
`git pull` download latest changes from tracked branch	`git pull`
`git pull origin <branch-name>` download latest changes from a specific branch	`git pull origin main`
`git branch` show local branches	`git branch`
`git branch --list` list branches	`git branch --list`
`git checkout <branch-name>` switch to a branch	`git checkout main`
`git checkout -b <branch-name>` create and switch to a new branch	`git checkout -b iszy-feature`
`git merge <branch-name>` merge another branch into current branch	`git merge iszy-feature`
`gh repo create <repo-name> --private --source=. --remote=origin` create a private GitHub repo from current folder	`gh repo create sample-app --private --source=. --remote=origin`
`gh repo view --web` open current GitHub repo in browser	`gh repo view --web`
`unset GITHUB_TOKEN` remove existing GitHub token variable	`unset GITHUB_TOKEN`
`unset GH_TOKEN` remove existing GH token variable	`unset GH_TOKEN`
`gh auth login --scopes repo` log in to GitHub CLI with repo permission	`gh auth login --scopes repo`
`gh auth setup-git` connect GitHub CLI login to Git	`gh auth setup-git`

Docker Commands

COMMAND	EXAMPLE
`docker --version` check Docker version	`docker --version`
`docker ps` show running containers	`docker ps`
`docker ps -a` show all containers	`docker ps -a`
`docker pull <image-name>` download an image	`docker pull jenkins/jenkins:lts`
`docker images` list local images	`docker images`
`docker build -t <image-name> .` build image from Dockerfile in current folder	`docker build -t sampleapp .`
`docker build -t <username>/<image-name>:<tag> .` build and tag image for Docker Hub	`docker build -t <your-dockerhub-username>/sampleapp:manual .`
`docker run -d -p <host-port>:<container-port> --name <container-name> <image-name>` run container in background with port mapping	`docker run -d -p 5050:5050 --name samplerunning sampleapp`
`docker run -t -d -p <host-port>:<container-port> --name <container-name> <image-name>` run container detached with TTY	`docker run -t -d -p 5050:5050 --name samplerunning sampleapp`
`docker run -it <image-name> <command>` run container interactively	`docker run -it ubuntu bash`
`docker stop <container-name>` stop a running container	`docker stop samplerunning`
`docker rm <container-name>` remove a stopped container	`docker rm samplerunning`

COMMAND	EXAMPLE
`docker rm -f <container-name>` force remove a container	`docker rm -f samplerunning`
`docker stop <container-name> && docker rm <container-name>` stop and remove container	`docker stop samplerunning && docker rm samplerunning`
`docker exec -it <container-name> <command>` run command inside a container	`docker exec -it jenkins_server cat /var/jenkins_home/secrets/initialAdminPassword`
`docker network inspect bridge | grep Gateway` find Docker bridge gateway IP	`docker network inspect bridge | grep Gateway`
`docker login -u <dockerhub-username>` log in to Docker Hub	`docker login -u iszydev`
`docker tag <old-image> <new-image>:<tag>` tag an image	`docker tag <your-dockerhub-username>/sampleapp:manual <your-dockerhub-username>/sampleapp:v1`
`docker push <username>/<image-name>:<tag>` upload image to Docker Hub	`docker push <your-dockerhub-username>/sampleapp:manual`
`docker logout` log out from Docker Hub	`docker logout`
`docker ps -q --filter publish=<port> | xargs -r docker rm -f` remove container using a specific published port	`docker ps -q --filter publish=5050 | xargs -r docker rm -f`

Jenkins / CI-CD Commands

COMMAND
```
docker run --rm -u root -p <host-port>:<container-port> \
-v <jenkins-volume>:/var/jenkins_home \
-v <docker-path>:/usr/bin/docker \
-v /var/run/docker.sock:/var/run/docker.sock \
-v "$HOME":/home \
--name <container-name> <image-name>
```
run Jenkins container with Docker access

EXAMPLE
```
docker run --rm -u root -p 8080:8080 \
-v jenkins-data:/var/jenkins_home \
-v /usr/bin/docker:/usr/bin/docker \
-v /var/run/docker.sock:/var/run/docker.sock \
-v "$HOME":/home \
--name jenkins_server jenkins/jenkins:lts
```

COMMAND
```
docker exec -it <container-name> cat
<password-path>
```
get Jenkins first admin password

EXAMPLE
```
docker exec -it jenkins_server cat
/var/jenkins_home/secrets/initialAdminPassword
```

COMMAND
```
curl http://<ip-address>:<port>/
```
test a web app URL

EXAMPLE
```
curl http://172.17.0.1:5050/
```

COMMAND
```
curl http://<ip-address>:<port>/ | grep "<text>"
```
test that a page contains expected text

EXAMPLE
```
curl http://172.17.0.1:5050/ | grep "You are
calling me from"
```

COMMAND
```
if curl http://<ip-address>:<port>/ | grep "<text>"; then
exit 0
else
exit 1
fi
```
Jenkins shell test pattern: pass = 0, fail = 1

EXAMPLE
```
if curl http://172.17.0.1:5050/ | grep "You are calling me
from"; then
exit 0
else
exit 1
fi
```

COMMAND
```
bash ./<script-name>.sh
```
run a Bash script

EXAMPLE
```
bash ./sample-app.sh
```

COMMAND
```
echo "<text>" >> <file-path>
```
append text to a file

EXAMPLE
```
echo "FROM python" >> tempdir/Dockerfile
```

Dockerfile Commands

COMMAND
```
FROM <base-image>
```
choose starting image

EXAMPLE
```
FROM python
```

COMMAND
```
RUN <command>
```
run command during image build

EXAMPLE
```
RUN pip install flask
```

COMMAND
```
COPY <source> <destination>
```
copy files into image

EXAMPLE
```
COPY sample_app.py /home/myapp/
```

COMMAND	EXAMPLE
`WORKDIR <directory-path>` set working directory	`WORKDIR /home/myapp`
`EXPOSE <port-number>` document app port	`EXPOSE 5050`
`CMD <command>` command to run when container starts	`CMD python3 /home/myapp/sample_app.py`

Python / Testing Commands

COMMAND	EXAMPLE
`python3 <file-name>.py` run a Python file	`python3 sample_app.py`
`pip install <package-name>` install Python package	`pip install flask`
`python -m unittest <test-file-name>.py` run unittest tests	`python -m unittest test_sample.py`
`pytest <test-file-name>.py` run pytest tests	`pytest test_sample.py`

Dockerfile - Placeholder Version

Use this when the exam gives different file names, ports, folders, or start commands.
Replace anything inside < > with the value from your question or lab.

Where to put it:	Create a file named Dockerfile at the root of <project-folder>.
Why:	docker build . looks for this file in the current folder.

Dockerfile Template with Placeholders

```
FROM <base-image>
RUN <install-command>
COPY <local-folder-or-file> <container-folder>/
COPY <local-folder-or-file> <container-folder>/
COPY <app-file> <container-app-folder>/
EXPOSE <app-port>
CMD <start-command>
```

Example Filled In

```
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 5050
CMD python3 /home/myapp/sample_app.py
```

What Each Placeholder Means

Placeholder	What to put	Example
`<base-image>`	base image / runtime for app	`python`
`<install-command>`	command to install dependencies	`pip install flask`
`<local-folder-or-file>`	file/folder in your repo	`./static`
`<container-folder>`	folder path inside image	`/home/myapp/static`
`<app-file>`	main app file name	`sample_app.py`
`<container-app-folder>`	where app file is copied inside image	`/home/myapp`
`<app-port>`	port the app listens on	`5050`
`<start-command>`	command to start app when container runs	`python3 /home/myapp/sample_app.py`

Keep Dockerfile keywords the same: FROM, RUN, COPY, EXPOSE, CMD. Only replace the <placeholders>.

Jenkins Pipeline Syntax

COMMAND

```
node {
stage('<stage-name>') {
<steps>
}
}
```

basic scripted pipeline shape

EXAMPLE

```
node {
stage('Build') {
build 'BuildAppJob'
}
}
```

COMMAND

```
stage('<stage-name>') {
<steps>
}
```

create one pipeline stage

EXAMPLE

```
stage('Preparation') {
sh 'docker stop samplerunning'
}
```

COMMAND

```
sh '<shell-command>'
```

run shell command in Jenkins

EXAMPLE

```
sh 'docker stop samplerunning'
```

COMMAND

```
build '<job-name>'
```

run another Jenkins job

EXAMPLE

```
build 'BuildAppJob'
```

COMMAND

```
catchError(buildResult: 'SUCCESS') {
<steps>
}
```

do not fail pipeline if cleanup command errors

EXAMPLE

```
catchError(buildResult: 'SUCCESS') {
sh 'docker stop samplerunning'
sh 'docker rm samplerunning'
}
```

COMMAND

```
pipeline {
agent any
stages {
stage('<stage-name>') {
steps {
sh '<command>'
}
}
}
}
```

basic declarative pipeline shape

EXAMPLE

```
pipeline {
agent any
stages {
stage('Build') {
steps {
sh 'docker build -t sampleapp .'
}
}
}
}
```

COMMAND

```
environment {
<VARIABLE_NAME> = '<value>'
}
```

set pipeline variables

EXAMPLE

```
environment {
DOCKER_USER = 'your-dockerhub-username'
IMAGE_NAME = "${DOCKER_USER}/sampleapp"
IMAGE_TAG = "${BUILD_NUMBER}"
}
```

COMMAND

```
triggers {
pollSCM('<schedule>')
}
```

check GitHub for updates on a schedule

EXAMPLE

```
triggers {
pollSCM('H/2 * * * *')
}
```

COMMAND

```
git branch: '<branch-name>',
credentialsId: '<credential-id>',
url: '<repository-url>'
```

Jenkins Git checkout

EXAMPLE

```
git branch: 'main',
credentialsId: 'github-cred',
url: 'https://github.com/<your-username>/sample-app-lesson-10.git'
```

One-Line Combo Commands

COMMAND	EXAMPLE

`mkdir -p <project-folder>/templates <project-folder>/static`
`cd <project-folder>`
create project folders and enter project
	`mkdir -p ~/sample-app/templates ~/sample-app/static`
	`cd ~/sample-app`

`git add . && git commit -m "<message>" && git push` stage, commit, and push quickly	`git add . && git commit -m "Update heading" && git push`
`docker stop <container-name> && docker rm <container-name>` stop and remove a container	`docker stop samplerunning && docker rm samplerunning`
`docker --version && docker ps` check Docker works	`docker --version && docker ps`

Lab 3.3.11 - Software Version Control with Git

COMMAND	EXAMPLE
`git config --list` show current Git configuration	`git config --list`
`ls -a` show hidden files such as .git	`ls -a`
`ls -la` show detailed file list including permissions	`ls -la`
`echo "<text>" > <file-name>` create/overwrite a file with text	`echo "I am on my way to passing the Cisco DEVASC exam" > DEVASC.txt`
`echo "<text>" >> <file-name>` append text to the end of a file	`echo "I am beginning to understand Git!" >> DEVASC.txt`
`git log` show commit history	`git log`
`git diff <old-commit-id> <new-commit-id>` compare two commits	`git diff b510f8e 9f5c4c5`
`git branch <branch-name>` create a new branch	`git branch feature`
`git checkout <branch-name>` switch to an existing branch	`git checkout feature`
`git merge <branch-name>` merge another branch into the current branch	`git merge feature`
`git branch -d <branch-name>` delete a branch after merging	`git branch -d feature`
`git commit -a -m "<message>"` commit all tracked modified files without separate git add	`git commit -a -m "Change Cisco to NetAcad"`

COMMAND	EXAMPLE
`sed -i 's/<old-text>/<new-text>/' <file-name>` replace text inside a file from terminal	`sed -i 's/Cisco/NetAcad/' DEVASC.txt`
COMMAND `vim <file-name>` open a file in Vim to fix/edit manually	**EXAMPLE** `vim DEVASC.txt`
COMMAND `cp ../<file-name> .` copy a file from parent folder into current folder	**EXAMPLE** `cp ../DEVASC.txt .`
COMMAND `git remote --verbose` check remote fetch/push URL	**EXAMPLE** `git remote --verbose`
COMMAND `git remote rm origin` remove wrong GitHub remote URL	**EXAMPLE** `git remote rm origin`
COMMAND `git push origin master` push local master branch to GitHub	**EXAMPLE** `git push origin master`

Lab 6.2.7 - Build a Sample Web App in a Docker Container

COMMAND	EXAMPLE
`touch <file-name>` create an empty file	`touch user-input.sh`
`nano <file-name>` open file in nano editor	`nano user-input.sh`
`#!/bin/bash` shebang line for a bash script	`#!/bin/bash`
`echo -n "<prompt-text>"` print prompt without a new line	`echo -n "Enter Your Name: "`
`read <variable-name>` read user input into a variable	`read userName`
`echo "<text> $<variable-name>"` print text and variable value	`echo "Your name is $userName."`
`bash <script-name>.sh` run a bash script with bash	`bash user-input.sh`
`chmod a+x <script-name>` make script executable for all users	`chmod a+x user-input.sh`
`mv <old-name> <new-name>` rename or move a file	`mv user-input.sh user-input`
`./<script-name>` run executable script from current folder	`./user-input`
`pip3 install <package-name>` install a Python package	`pip3 install flask`
`python3 <file-name>.py` run a Python file	`python3 sample_app.py`

COMMAND	EXAMPLE
`curl http://<ip-address>:<port>` test a web server from terminal	`curl http://0.0.0.0:8080`
`cat templates/index.html` view the HTML file	`cat templates/index.html`
`cat static/style.css` view the CSS file	`cat static/style.css`
`echo "FROM python" > tempdir/Dockerfile` create Dockerfile and set base image	`echo "FROM python" > tempdir/Dockerfile`
`echo "RUN pip install flask" >> tempdir/Dockerfile` add Flask install step to Dockerfile	`echo "RUN pip install flask" >> tempdir/Dockerfile`
`echo "EXPOSE <port>" >> tempdir/Dockerfile` add exposed port to Dockerfile	`echo "EXPOSE 8080" >> tempdir/Dockerfile`
`echo "CMD <command>" >> tempdir/Dockerfile` add startup command to Dockerfile	`echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile`
`docker run -t -d -p <host-port>:<container-port> --name <container-name> <image-name>` run container in background with terminal and port mapping	`docker run -t -d -p 8080:8080 --name samplerunning sampleapp`
`docker ps -a >> <file-name>` save container list output into a file	`docker ps -a >> running.txt`
`ip address` show network interfaces and IP addresses	`ip address`
`docker exec -it <container-name> /bin/bash` enter a running container shell	`docker exec -it samplerunning /bin/bash`
`docker start <container-name>` start an existing stopped container	`docker start samplerunning`

Lab 6.3.6 - Build a CI/CD Pipeline Using Jenkins

COMMAND	EXAMPLE
`cd labs/devnet-src/jenkins/sample-app/` go to the Jenkins sample-app folder	`cd labs/devnet-src/jenkins/sample-app/`
`git add *` stage all files in the current folder	`git add *`
`git commit -m "Committing sample-app files."` commit sample app files	`git commit -m "Committing sample-app files."`
`git push origin master` push sample app to GitHub master branch	`git push origin master`
`git commit -m "Changed port from 8080 to 5050."` commit the port change before Jenkins uses 8080	`git commit -m "Changed port from 8080 to 5050."`
`docker pull jenkins/jenkins:lts` download Jenkins LTS image	`docker pull jenkins/jenkins:lts`
`docker run --rm -u root -p 8080:8080 -v <jenkins-volume>:/var/jenkins_home -v $(which docker):/usr/bin/docker -v /var/run/docker.sock:/var/run/docker.sock -v "$HOME":/home --name <container-name> <image-name>` run Jenkins with Docker access	`docker run --rm -u root -p 8080:8080 -v jenkins-data:/var/jenkins_home -v $(which docker):/usr/bin/docker -v /var/run/docker.sock:/var/run/docker.sock -v "$HOME":/home --name jenkins_server jenkins/jenkins:lts`
`docker exec -it <container-name> /bin/bash` open shell inside Jenkins container	`docker exec -it jenkins_server /bin/bash`
`bash ./sample-app.sh` Jenkins build command for BuildAppJob	`bash ./sample-app.sh`
`docker stop <container-name>` stop old app container before test/pipeline	`docker stop samplerunning`
`docker rm <container-name>` remove old app container before test/pipeline	`docker rm samplerunning`

COMMAND	EXAMPLE
`curl http://<gateway-ip>:5050/ | grep "You are calling me from"` test if sample app page contains expected text	`curl http://172.17.0.1:5050/ | grep "You are calling me from"`
COMMAND `if curl http://<gateway-ip>:5050/ | grep "<expected-text>"; then exit 0; else exit 1; fi` Jenkins TestAppJob shell pattern	**EXAMPLE** `if curl http://172.17.0.1:5050/ | grep "You are calling me from"; then exit 0; else exit 1; fi`
COMMAND `node { stage('<stage-name>') { <steps> } }` basic scripted Jenkins pipeline structure	**EXAMPLE** `node { stage('Build') { build 'BuildAppJob' } }`
COMMAND `catchError(buildResult: 'SUCCESS') { <steps> }` ignore cleanup error so pipeline can continue	**EXAMPLE** `catchError(buildResult: 'SUCCESS') { sh 'docker stop samplerunning'; sh 'docker rm samplerunning' }`
COMMAND `build '<job-name>'` run another Jenkins job from pipeline	**EXAMPLE** `build 'TestAppJob'`
COMMAND `sh '<command>'` run shell command from Jenkins pipeline	**EXAMPLE** `sh 'docker rm samplerunning'`